

Data Structure and Algorithms

Searching



What is Searching in Data Structure?

Searching means to find whether a particular data item exists in an array/list or not. The process of finding a particular value in a list or an array is called **searching**.

If that particular value is present in the array, then the search is said to be successful and the location of that particular value is returned by the searching process. However, if the value does not exist then searching is said to be **unsuccessful**.

The process of finding the desired information from the set of items stored in the form of elements in the computer memory is referred to as 'searching in data structure'.

These sets of items are in various forms, such as an array, tree, graph, or linked list. Another way of defining searching in the data structure is by locating the desired element of specific characteristics in a collection of items.

There are many different searching algorithms, but three of the popular searching techniques are as follows:

- Linear Search or Sequential Search
- Binary Search
- Interpolation Search



1. Linear search or Sequential Search

A linear search is also called a sequential search. This is a very simple technique used to search for a particular value in an array.

A linear search works by comparing the value of the key being searched for with every element of the array in a linear sequence until a match is found.

- The linear search is the simplest way to search for an element in a list.
- The linear searches the data element sequentially, no matter whether the array is sorted or unsorted.

For example- Let us take an array of ten elements, which is declared as follows:

```
int array[10] = {87,25,14,39,74,1,99,12,30,67};
```

and the value to be searched for in the array is VALUE = 74, and then search to find whether 74 exists in the array or not. If the value is present then its

position is returned as well



Implementation of Linear search using arrays.

Let ARR be an array of n elements, ARR[1], ARR[2], ARR[3], ...ARR[n] Such that VAL is the element to be searched. Then the algorithm will find the position POS of the VAL in the array ARR.

```
Step 1: START
Step 2: Set I = 0, POS = -1
Step 3: Repeat while I<N
            IF (ARR[I] == VAL)
                POS = I
            PRINT POS
            Go to Step 4
[End of IF]
[End of Loop]
Step 4: IF (POS = -1)
            PRINT "VALUE NOT FOUND, SEARCH
UNSUCCESSFUL"
            [End of IF]
Step 5: EXIT
```



Implementation of Linear Search

```
#include<bits/stdc++.h>
using namespace std;

int linearSearch(int a[],int n,int key){

    for(int i=0;i<n;i++){

        if(a[i] == key){
            return i;
        }
    }

    return -1;

}
```

```
int main(){
    int n;
    cin>>n;
    int a[n];
    for(int i=0;i<n;i++){
        cin>>a[i];
    }
    cout<<"enter key"<<endl;
    int key;
    cin>>key;

    int pos = linearSearch(a,n,key);
    if(pos == -1){
        cout<<"key not found"<<endl;
    }else{
        cout<<"key found at index "<<pos<<endl;
    }

    return 0;
}
```



Drawback of Linear Search

1. It is a very time-consuming process, as it works sequentially.
2. It can be applied only to a small amount of data.
3. It is a very slow process as almost every data element is accessed in this process, especially when the data element is located near the end.



Binary Search

Binary search is a searching algorithm that finds the position for an element in a sorted array. Binary search begins by comparing an element in the middle of the array with the target value. If the target value matches the element, its position in the array is returned.

A binary search is an extremely efficient searching algorithm when it is compared to a linear search. A binary search works only when the array/list is already sorted.

If the target value is less than the element, the search continues in the lower half of the array. If the target value is greater than the element, the search continues in the upper half of the array. By doing this, the algorithm eliminates the half in which the target value cannot lie in each iteration



Binary Search

In a binary search, we first compare the value VAL with the data element in the middle position of the array.

If the match is found, then the position POS of that element is returned; otherwise, if the value is less than that of the middle element, then we begin our search in the lower half of the array and vice versa.

So, we repeat this process on the lower and upper half of the array until a match is found or else the array partition is reduced to empty . For example, given any array:

1. Find the middle element of the array, that is, $n/2$ is the index of the middle element of the array containing n elements.
2. Now, compare the middle element of the array with the data element to be searched.
 - a. If the middle element is the desired element, then the search is successful.
 - b. If the data element to be searched for is less than the middle element of the array, then search only the lower half of the array, that is those elements that are on the left side of the middle element.
 - c. If the data element to be searched for is greater than the middle element of the array, then search only the upper half of the array, that is, those elements which are on the right side of the middle element.



Algorithm for a binary search

Step 1: START

Step 2: Set $BEG = \text{lower-bound}$, $END = \text{upper_bound}$, $POS = -1$

Step 3: Repeat Steps 4 & 5 **while** $BEG \leq END$

Step 4: Set $MID = (BEG + END) / 2$

Step 5: IF ($ARR(MID) = VAL$)

$POS = MID$

 PRINT POS

 Go to Step 7

ELSE IF ($ARR(MID) > VAL$)

 Set $END = MID - 1$

ELSE

 Set $BEG = MID + 1$

 (End of If)

 [End of loop]

Step 6: IF ($POS = -1$)

 PRINT "VALUE NOT FOUND; SEARCH UNSUCCESSFUL"

 [End of IF]

Step 7: EXIT



Implementation of Binary Search

```
#include<bits/stdc++.h>
using namespace std;

int BinarySearch(int a[],int n,int key){

    int s = 0, e = n-1;
    while(s<=e){
        int mid = s + (e-s)/2;

        if(a[mid] == key){
            return mid;
        }else if(a[mid] > key){
            e = mid-1;
        }else{
            s = mid + 1;
        }
    }

    return -1;
}
```

```
int main(){

    int n;
    cin>>n;
    int a[n];
    for(int i=0;i<n;i++){
        cin>>a[i];
    }

    cout<<"enter key"<<endl;
    int key;
    cin>>key;

    sort(a,a+n);

    int pos = BinarySearch(a,n,key);
    if(pos == -1){
        cout<<"key not found"<<endl;
    }else{
        cout<<"key found at index "<<pos<<endl;
    }

    return 0;
}
```



Binary Search Time Complexity

If k is the number of steps, the binary search will stop when we have one element.

$$(n/2^k)=1$$

$$n=2^k$$

$$\log n = k \log 2$$

$$k = \log_2 n$$

$$k = \log n$$

The Time Complexity of Binary Search is $O(\log n)$.



Program C++ with recursion

```
#include <bits/stdc++.h>
using namespace std;

// A recursive binary search function. It returns
// location of x in given array arr[l..r] is present,
// otherwise -1
int binarySearch(int arr[], int l, int r, int x)
{
    if (r >= l) {
        int mid = l + (r - l) / 2;

        // If the element is present at the middle
        // itself
        if (arr[mid] == x)
            return mid;

        // If the element is smaller than mid, then
        // it can only be present in the left subarray
        if (arr[mid] > x)
            binarySearch(arr, l, mid - 1, x);

        // Else the element can only be present
        // in right subarray
        binarySearch(arr, mid + 1, r, x);
    }

    // We reach here when an element is not
    // present in array
    return -1;
}
```

```
int main(void)
{
    int arr[] = { 2, 3, 4, 10, 40 };
    int x = 10;
    int n = sizeof(arr) / sizeof(arr[0]);
    int result = binarySearch(arr, 0, n -
1, x);
    (result == -1) ? cout << "Element is not present
in array": cout << "Element is present at index
" << result<<endl;
    return 0;
}
```

OUTPUT:

```
Element is present at index
3
```



Practical Application

Have a second to guess this example 😊

A real-life application of a binary search is that when we search for a particular word in a dictionary, we first open the dictionary somewhere in the middle.

Now we will compare the desired word with the first word on that page.

If the desired word comes after the first word on an open page, then we will look in the second half of the dictionary; otherwise, we will look in the first half.

Now, we will again open a page in the second half and compare the desired word with the first word on that page, and the same process is repeated until we have found the desired word.

Drawbacks of a BINARY SEARCH

1. A binary search requires that the data elements in the array be sorted; otherwise, a binary search will not work.
2. A binary search cannot be used where there are many insertions and deletions of data elements in the array.

